TripNest AI: Hackathon Submission Summary (PS 12)

Version 1.0 · 2 November 2026. The six sections requested by the organisers. Everything below is implemented and tested.

1. Problem Statement Analysis

Problem. Popular tourist places in India are overcrowded at certain days and hours, while equally good, lesser-known places stay empty. Travellers have no easy way to ask: *which popular places are near me, how crowded will they be on my date, when should I go, and where can I go to escape the crowds?*

Who is affected. Tourists and families (long queues, poor experience, safety risk), destination authorities (over-capacity at fragile sites), and local communities near hidden places that miss the benefit of tourism.

What the solution must do. (1) Predict tourist crowd by date and time of day. (2) Recommend destinations using that prediction, not popularity alone. It must be usable without registration, honest about the data behind each number, and work on phones.

Constraints and assumptions. No verified public source of hourly visitor counts exists for Indian attractions, so the model is trained on a clearly labelled demonstration history. Capacities are assumptions. Crowd levels are shares of that assumed capacity.

2. Proposed Solution

TripNest AI, a location-first web app. 1. The visitor allows location (or types a city) and picks a date and an optional interest. 2. **Popular places near you**, each with a photo, distance, a green/yellow/red predicted-crowd chip and a best time. 3. **Best time to visit**: a four-part day timeline (early morning, morning, afternoon, evening) with the calmest period and an hour window, given only when the data supports it. 4. **Escape the Crowds**: lesser-known places predicted to be calm on that date, each with the reason it was chosen and, where one exists, the busier place it is a calmer alternative to. 5. **Interactive crowd map** with different markers for you, popular places and hidden gems, filters (All, Popular, Hidden gems, Low, Moderate, High) and popups. 6. A **destination page** with a prediction for a date and time range, an hourly chart with a likely range, a 7-day trend and "About this prediction" (source, freshness, uncertainty, limitations).

Only the two core features are in the product: crowd prediction and smart destination recommendation.

3. Proposed Methodology / Approach

1. **Data.** A catalogue of 84 destinations and 271 attractions covering all 36 states and union territories, with coordinates, categories, opening hours, assumed capacity and a popularity score. Crowd history is a labelled **demonstration** generator with a documented contract (`key, date, hour, count, rain`) so real gate or sensor counts can replace it.
2. **Model.** A gradient-boosted regressor (scikit-learn) predicts utilisation = people ÷ assumed capacity, in log space, from category, climate, hour, weekday, holiday, season, rain, popularity, capacity and opening-hours features. No attraction ID is used, so a new attraction gets forecasts without retraining.
3. **Evaluation.** Chronological train, validation and test split (no shuffling), compared with two baselines, with an 80% likely range from the test errors.
4. **Crowd levels.** Low below 40%, Moderate 40–70%, High 70–90%, At Capacity above 90% (stricter for ecologically sensitive sites). The screen shows three colours.
5. **Popular places.** Catalogue popularity at least 0.9, nearest first. **Hidden gems:** popularity at most 0.8 and predicted Low or Moderate, scored on crowd (0.40), interest (0.25), distance (0.20) and season (0.15). Popularity is **not** a score factor.

6. **Best time.** The calmest hour between 5 AM and 7 PM; withheld when fewer than 3 hours are open or the lowest crowd is still 70% or more.
7. **Build process.** Tests written first, real browser tests for the user flow, defects traced to causes, documents kept in step with the code.

4. Technologies and Tools to be Used

Layer	Technology
Backend	Python 3.14, FastAPI, SQLAlchemy 2, Alembic
Database	SQLite (development), PostgreSQL-ready (production)
ML	scikit-learn `HistGradientBoostingRegressor`, pandas, numpy, joblib
Frontend	HTML, CSS and vanilla JavaScript modules (no build step), Inter font
Maps and photos	Leaflet with OpenStreetMap tiles; Wikimedia Commons photographs stored with credit and licence
Testing	pytest, Playwright with Chrome
Deployment	Docker, GitHub, Render (free plan, live)
Security	Secrets only in environment variables, input validation, security headers with a content-security policy, location rounded to about 1 km, nothing about the user stored

5. Expected Outcomes

- Travellers see predicted crowd, best hour and calm alternatives for their own location and date, without signing up.
- Less pressure on over-visited places by pointing demand to calm, lesser-known ones.
- A working pipeline that can be retrained on real visitor counts with one table and one command.
- Measured, honest results on the demonstration data: test error (MAPE) **9.38%** against **31.79%** (same hour last week) and **27.45%** (weekday-hour average). These show the pipeline works; **they say nothing about accuracy on real crowds.**
- Evidence: **101 backend tests and 24 browser tests pass** (run 2 November 2026), including the full nine-step flow on four screen sizes.

6. Initial Prototype / Solution Design

Prototype status. A working application, run locally and committed to GitHub (`github.com/Aicodedd/tripnest-ai`). **Live demo:** https://tripnest-ai-1rug.onrender.com (Render free plan; it sleeps when idle, so the first load can take about 30 seconds). Checked on 2 November 2026: health, API and all 24 browser tests pass against it.

Design. Three pages: home (hero photograph and search panel), explore (popular places, best time, hidden gems, map), destination details. Visual style follows the supplied reference: white cards, blue primary, green/yellow/red crowd chips, responsive from phone to desktop.

Architecture. Browser (vanilla JS, Leaflet) → FastAPI (explore, prediction, recommendation and catalogue routers; prediction, explore and recommender services) → database and a stored model file.

Supporting documents (all in `docs/`, with PDFs): MRD (1 page), PRD (2 pages), epics, user stories, requirement traceability, system architecture, database schema, API documentation, AI/ML documentation, test plan, test results, deployment guide, user guide, judge demo guide.

Known limitations. Simulated crowd history; assumed capacities; straight-line distances; catalogue details and coordinates are indicative and should be reviewed; no load, PostgreSQL or accessibility-audit testing

(the live app uses a seeded SQLite file); Chrome-only browser tests; compare screen and typed destination search are not in the current interface.

Future scope. Real data feeds (gate, ticketing, sensors), weather and events as model inputs, road-distance routing, an authority view, itinerary and community features.